ENTREGA DE PRACTICA

Mercado Local API

Django REST Framework, usuarios por email y JWT

Repositorio	https://github.com/ismael789/django-project
Proyecto	Tienda en linea Mercado Local
Navegador	Google Chrome en Windows
Documentacion	http://127.0.0.1:8000/api/swagger/
Token de acceso	30 minutos (ACCESS_TOKEN_LIFETIME)
Campo extra	carrera

Esta evidencia documenta los endpoints solicitados y las correcciones de seguridad implementadas.

1. Swagger y endpoints

La documentacion OpenAPI se publica con el titulo Mercado Local API. La interfaz muestra usuarios, productos, categorias, token y datos personales.

Swagger UI - Mercado Local API 200 OK

```
Mercado Local API
Version: 1.0.0

GET    /api/schema/
POST   /api/token/
POST   /api/token/refresh/
GET    /api/usuarios/datos-personales/
GET    /api/usuarios/
POST   /api/usuarios/
PATCH  /api/usuarios/{id}/
GET    /api/products/
GET    /api/categories/
```

El campo carrera aparece en el esquema del usuario y password es write-only.

2. Obtencion del token

El usuario personalizado inicia sesion con correo electronico. El token de acceso se genera con SimpleJWT y su vigencia es de 30 minutos.

Obtener JWT 200 OK

```
Request
{
  "email": "ana@example.com",
  "password": "ClaveSegura123"
}

Response 200 OK
{
  "refresh": "eyJ...token_de_refresco...",
  "access": "eyJ...token_de_acceso..."
}
```

Para las siguientes consultas se envia Authorization: Bearer <access>.

3. Datos personales protegidos

El endpoint requiere autenticacion JWT y devuelve los datos del usuario autenticado. El hash de password no forma parte de la respuesta.

Consulta autenticada 200 OK

```
Authorization: Bearer eyJ...token_de_acceso...

Response 200 OK
{
  "id": 1,
  "email": "ana@example.com",
  "first_name": "Ana",
  "last_name": "Lopez",
  "carrera": "Ingenieria"
}

password: ausente
pbkdf2: ausente
```

La respuesta usa PersonalDataSerializer sin campo password.

4. Actualizacion sin password

Editar otros datos no cambia la contraseña existente. El serializer solo ejecuta set_password cuando password esta presente.

Editar usuario sin contraseña 200 OK

```
Request
{
  "carrera": "Arquitectura"
}

Response 200 OK
{
  "id": 1,
  "email": "ana@example.com",
  "carrera": "Arquitectura"
}

Verificacion posterior
hash anterior == hash posterior: True
check_password("ClaveSegura123"): True
password en respuesta: ausente
```

Pruebas automaticas: 11 pruebas ejecutadas correctamente. Tambien se verificaron migraciones y OpenAPI.

Archivos relevantes

products/models.py | products/serializers.py | products/views.py | products/tests.py | requirements.txt | .gitignore